Joseph Dye, Mason Thomas, Simon Glashauser, Nick Russell

CSC 2400-001

4 April 2026

C. Radian

Exam Scheduling Problem – Second Checkup

Our exam scheduling solution encompassed four algorithmic approaches: a brute-force exhaustive approach, a greedy algorithm, a graph-coloring algorithm, and a genetic algorithm. In addition to creating these algorithms, we have also worked on synthesizing data that fits our use case and is in a standardized format as well as a schedule checker to verify results are accurate.

Coding these algorithms relied heavily on the use of Artificial Intelligence through models such as Claude, ChatGPT, and Copilot. However, AI is not perfect, so often manual adjustments were needed to yield a desired output. Each algorithm creates a valid solution, but their techniques differ.

The brute force exhaustive search generates all possible permutations of schedules based on JSON files of student, course, and location data over twenty possible timeslots. It stores every possible schedule, even the invalid ones, so long as each exam is in a unique timeslot. It then checks for conflicts, detecting if, for every student, a student is scheduled for two exams at the same time. Deleting all invalid schedules, the algorithm is left only with the valid, and it prints the first valid schedule in this list to the output file.

The greedy algorithm takes in the same types of input (JSON data files). It then builds a roster for each course, keeping track of which students are scheduled for each exam, and begins sorting by which exams the most students have assigned to them. Then, for every exam, working down from most to least populated, the algorithm checks every unused timeslot and selects the timeslot that would create the least amount of student conflicts. It always assigns an exam, meaning conflicts could still happen. The greedy algorithm then writes one complete schedule to an output file. This algorithm runs very fast, but it could still potentially have conflicts.

The graph-coloring algorithm takes similar JSON inputs and then creates a student course list and course rosters. A conflict graph is then built where each node is an exam and an edge between two exams indicates a shared student. This models the idea that if two exams have an edge, they should not be scheduled at the same time. Assigning exams follows a degree of saturation (DSatur) coloring technique that picks the exam with the highest “saturation,”

meaning the exam with the most restricted choices, or the most conflicts, and assigns it to a timeslot. From there, time-slotted exams are given a location, and the schedule is output to a file.

The genetic algorithm represents data via chromosomes (a full exam schedule) and genes (an exam's timeslot). The algorithm reads in JSON input files as well as genetic information regarding how many generations to have, population size, etc. A fitness function is used that starts at a perfect fitness level of ten-thousand and subtracts from that number based on student and location conflicts. The algorithm then selects a small pool of schedules and keeps the best of that group, later crossing it over with another schedule (from another pool), and combines their schedules. Random mutation is introduced to randomly adjust some exam timeslots, and a fitness score is reevaluated. Elitism is used to keep the best solution found so far, and the process repeats itself. The algorithm continues until a perfect fitness score is reached or the generation count is exceeded, and it then saves the best solution to an output file.

All results are generated in the same format so that the schedule checking algorithm can be used. Regarding input data, we decided to use a fully synthesized approach due to the abstract nature of the necessary data. Furthermore, student data was not easily available, as which classes every student takes at a university is not made public, so some level of artifice was needed. What is important is the quantity of our data, and we determined it as follows. Starting with four exams, we doubled exam size until we reached 2^{16} exams. Deciding that each student likely took around five classes, the size of the student files are five times the size of the associated course file. The number of locations follows the idea that there are twenty timeslots (four times a day, five times a week) so the number of locations needs to be equal to the number of exams divided by twenty. For exam counts less than or close to twenty, this equation will not work, so location sizes four, eight, and sixteen files were also created.

Our experience thus far with creating algorithms has been a success. One point of future concern, however, is gathering meaningful runtime data. Often the brute-force technique is ridiculously long, to the point repetition is not feasible, as with our smallest current input size it generates $\frac{80!}{76!}$ permutations. On the other hand, the greedy and graph coloring algorithms are so fast it may take infeasible input sizes to get a large enough running time difference. The genetic algorithm is a bit of a wild card in terms of running time, as, if a solution exists, it can usually find it quickly, however, if no solution exists, it must exhaust all generations before a decision is made.